KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Artificial Intelligence and Data Science

ASSIGNMENT II — CASE STUDY

Exploratory Data Analysis and Deep Learning for Telecom Customer Churn

Submitted by: PRITHEESHKUMAR V

Register Number: 25ADL07

Department: Artificial Intelligence and Data Science

Year / Semester: V Semester

Course Code: U21ADP05

Course Title: Exploratory Data Analysis and Visualization

Course Coordinator: Mr. Rushikesh Kadam

Academic Year: 2026–2027 (Odd Semester)

Abstract

Customer churn is an important business problem for telecommunications providers because customers who discontinue services can reduce recurring revenue and increase replacement costs. This case study uses the public IBM Telco Customer Churn sample dataset to explore customer demographics, subscribed services, contract arrangements, tenure, and billing attributes. The workflow includes data type correction, missing-value handling, duplicate checks, exploratory statistical summaries, and visualizations of target balance, tenure, monthly charges, contract type, and numerical correlations. Categorical variables are one-hot encoded and numeric features are standardized in a preprocessing pipeline. A Multilayer Perceptron (MLP) classifier is trained and selected using validation ROC-AUC, then evaluated on a held-out test set using accuracy, precision, recall, F1-score, ROC-AUC, and supporting charts. Final numerical findings must be inserted from the reproducible script output rather than assumed in advance.

1. Introduction

Telecommunication services often operate through recurring subscriptions. When a customer leaves, the provider may lose future revenue and may need to spend resources acquiring a replacement customer. Data analysis can help identify service, billing, and contract patterns associated with churn. This project combines exploratory data analysis (EDA), visualization, and a basic deep learning classifier to investigate this practical problem.

1.1 Objectives

- Explore a real-world dataset with at least 15 fields.
- Inspect data types, missing values, duplicates, distributions, and possible outliers.
- Create more than five interpretable visualizations and explain the purpose of each.
- Prepare numerical and categorical features for modeling and create train, validation, and test splits.
- Train an MLP classifier, compare a small set of hyperparameter choices, and evaluate performance.
- Document limitations, practical recommendations, and future scope.

2. Dataset Description

The case study uses IBM's public Telco Customer Churn sample dataset. The fields represent customer demographics, account tenure, subscribed services, contract and payment arrangements, charges, and a binary Churn target. The customerID field uniquely identifies records and is excluded from model inputs. The exact number of rows, columns, missing values, and descriptive statistics are printed by the program at runtime and should be copied into the final version after execution.

Dataset source: https://github.com/IBM/telco-customer-churn-on-icp4d

3. Methodology and Preprocessing

Loading

The Python script downloads the CSV from the public IBM sample repository when the local file is not present.

Cleaning

TotalCharges is converted to numeric, invalid blank strings become missing values, duplicate rows are removed, and rows without a target label are excluded.

Missing values

Numeric features use median imputation and categorical features use most-frequent imputation inside the model pipeline. The original missing-value counts are also printed for inspection.

Encoding and scaling

Categorical features are one-hot encoded with unknown categories ignored. Numeric features are median-imputed and standardized.

Data splitting

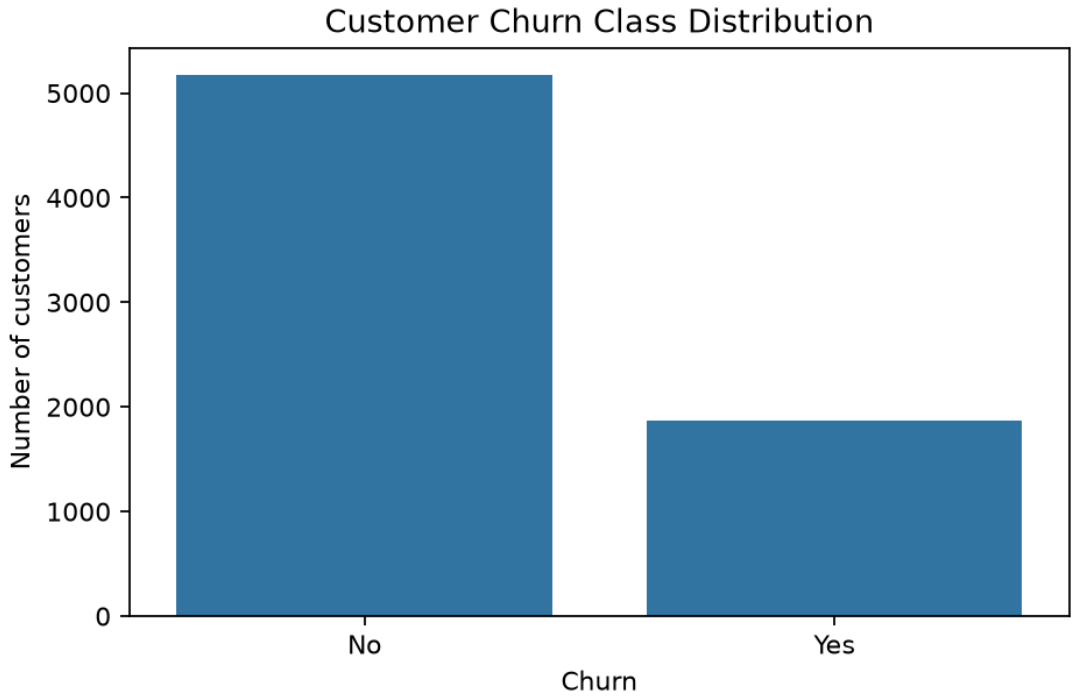
A stratified test set is held out first. The remaining training data is split again to create a validation set. The selected model is refit on the full training-plus-validation portion before final test evaluation.

Outlier analysis

The monthly-charge boxplot and numeric summaries support visual inspection of spread and potential outliers. Extreme values are not automatically deleted because high charges can be valid customer behavior.

4. Exploratory Data Analysis and Visualization

Figure 1 — Churn class distribution

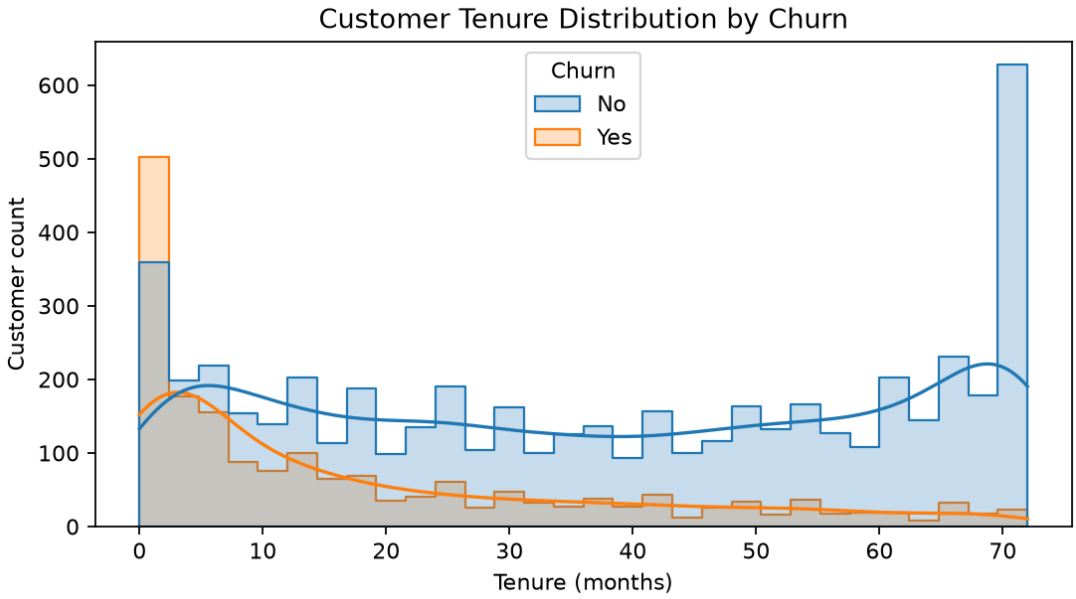


Inserted chart: 01_churn_distribution.png

The dataset contains 5,174 customers who did not churn and 1,869 customers who churned. Churn represents approximately 26.54% of all records.

Shows the number of customers in the Churn and No Churn classes. It is used to assess target imbalance, which matters when interpreting accuracy and recall.

Figure 2 — Tenure distribution by churn

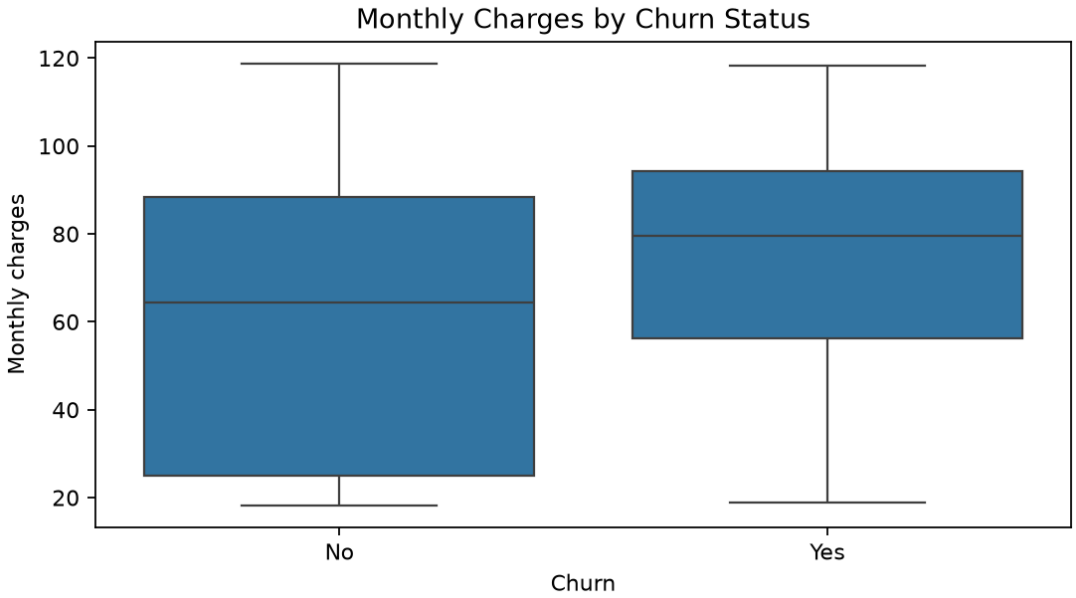


Inserted chart: 02_tenure_distribution.png

This histogram compares customer tenure for churned and retained customers. Use the plotted distributions to discuss how tenure differs between the two groups.

Compares customer tenure distributions for churned and retained customers. It helps explore whether churn is more common among newer or longer-tenure customers.

Figure 3 — Monthly charges by churn

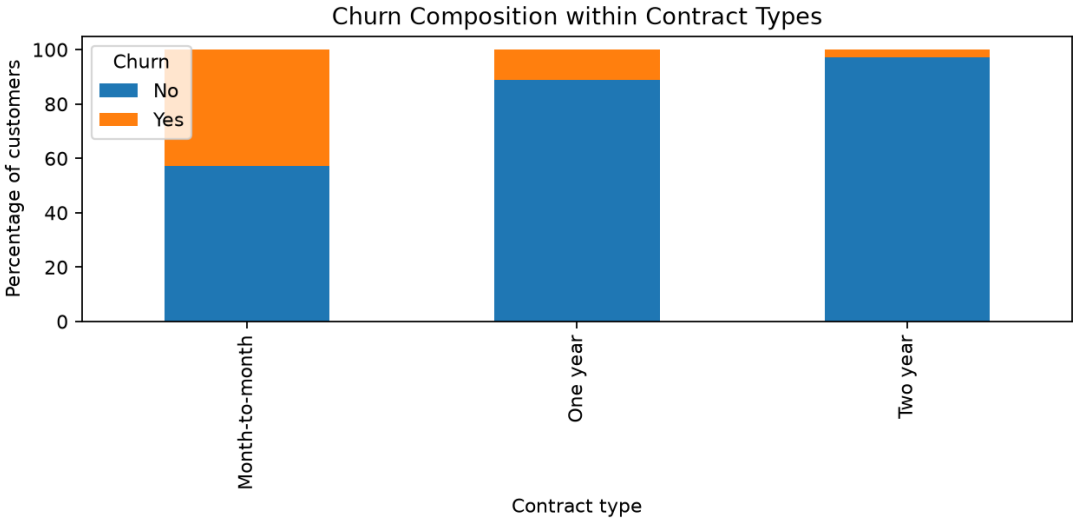


Inserted chart: 03_monthly_charges_boxplot.png

This boxplot compares the median, spread, and potential extreme monthly charges for churned and retained customers.

A boxplot compares the median, spread, and potential extreme monthly charges for each churn group.

Figure 4 — Contract type and churn

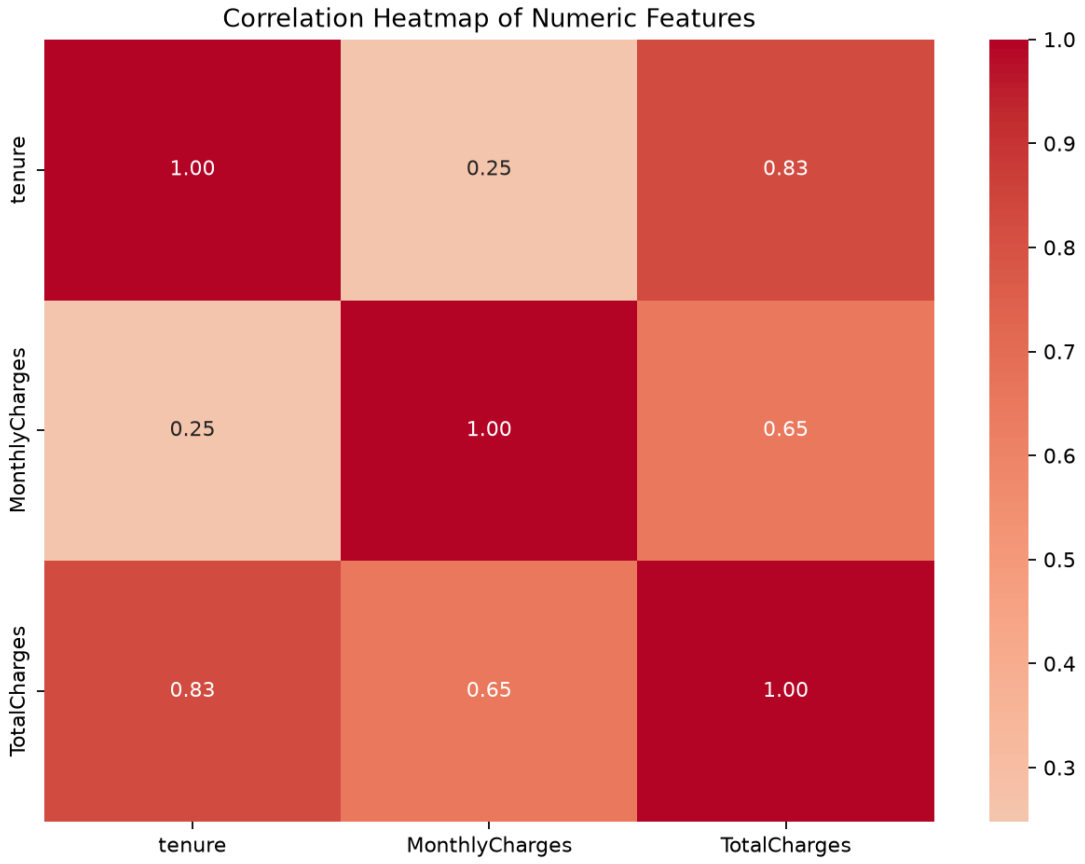


Inserted chart: 04_contract_churn.png

This chart compares churn composition across contract types. Discuss visible differences between month-to-month, one-year, and two-year contracts without treating association as proof of causation.

A normalized stacked bar chart compares churn composition across month-to-month, one-year, and two-year contracts. It helps identify associations between contract commitment and churn.

Figure 5 — Numeric correlation heatmap



Inserted chart: 05_correlation_heatmap.png

The heatmap displays pairwise correlations among numeric variables. Correlation indicates association and does not establish causation.

Displays pairwise correlations among numeric fields. Correlation indicates linear association, not causation.

Figure 6 — Internet service and churn

Compares churn percentages across internet service categories to identify potential differences between customer groups.

The charts are generated in outputs/figures by running the script. Insert the actual saved charts and add one or two observations based on the figures before submission. Do not report a trend unless it is visible in the generated result.

5. Deep Learning Model

A Multilayer Perceptron (MLP) classifier is used as a feed-forward neural network for binary classification. The preprocessing pipeline transforms raw numeric and categorical data into model-ready features. ReLU activation and the Adam optimizer are used. A small candidate set of hidden-layer structures and L2 regularization strengths is compared using validation ROC-AUC. Early stopping is enabled to reduce unnecessary training iterations.

5.1 Model inputs and target

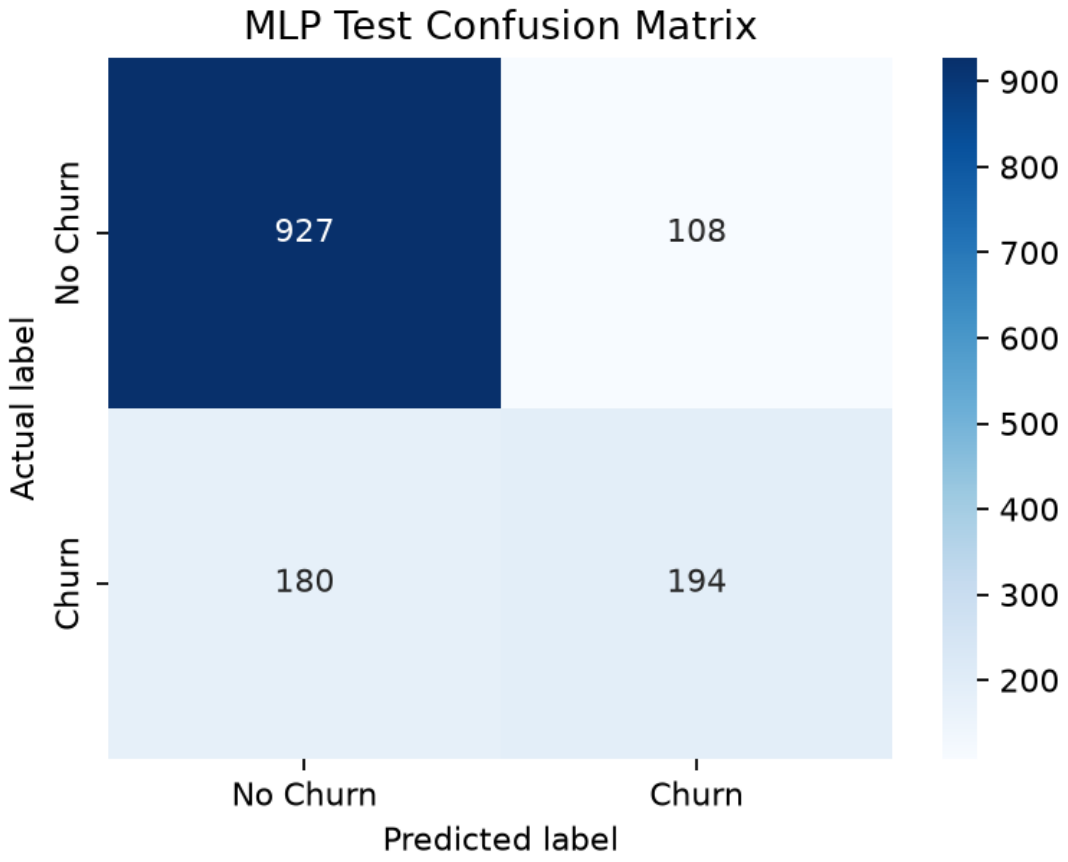
Inputs: all available customer attributes except customerID. Target: Churn, mapped to 1 for Yes and 0 for No.

5.2 Evaluation metrics

- Accuracy: proportion of all predictions that are correct.
- Precision: proportion of predicted churners who actually churned.
- Recall: proportion of actual churners correctly identified.
- F1-score: harmonic mean of precision and recall.
- ROC-AUC: ranking ability across decision thresholds; 0.5 is approximately random ranking and 1.0 is perfect ranking.

6. Model Result Visualization and Interpretation

Figure 7 — Confusion matrix

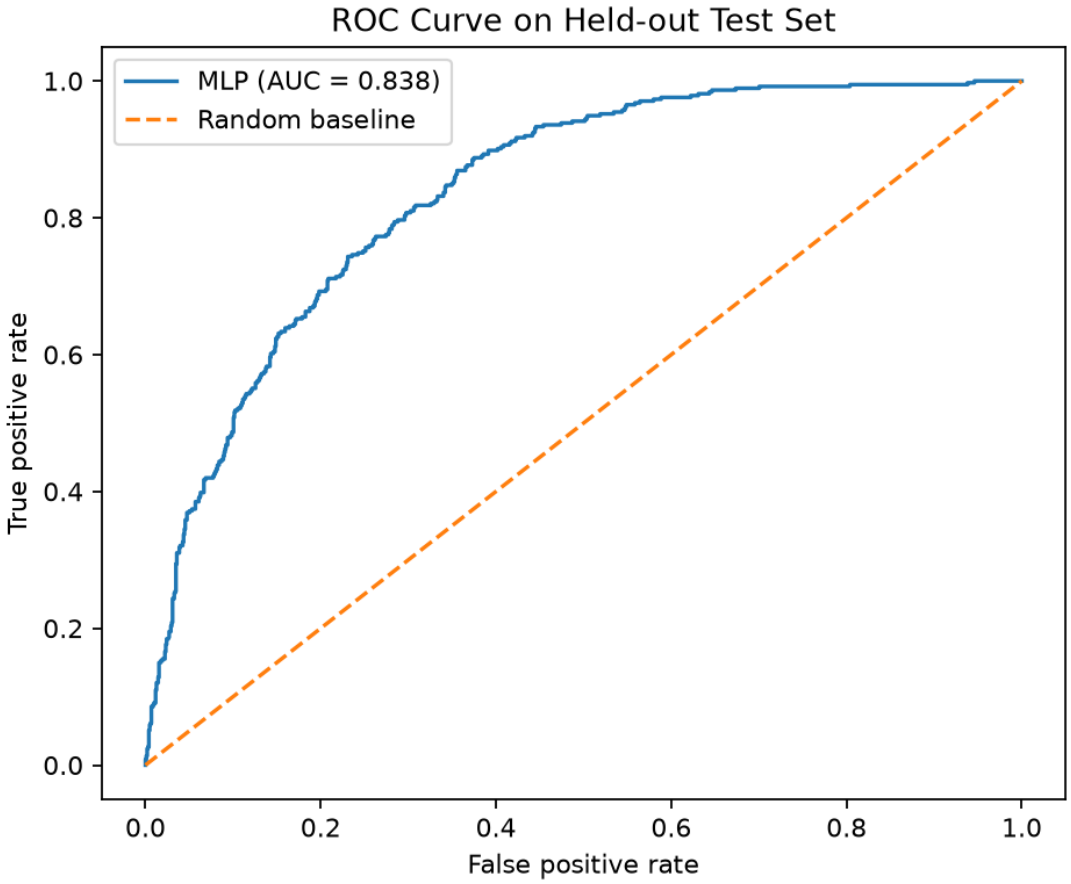


Inserted chart: 07_confusion_matrix.png

The confusion matrix summarizes correct and incorrect predictions for the held-out test set. It helps distinguish false positives from missed churners.

Counts true negatives, false positives, false negatives, and true positives on the held-out test set.

Figure 8 — ROC curve

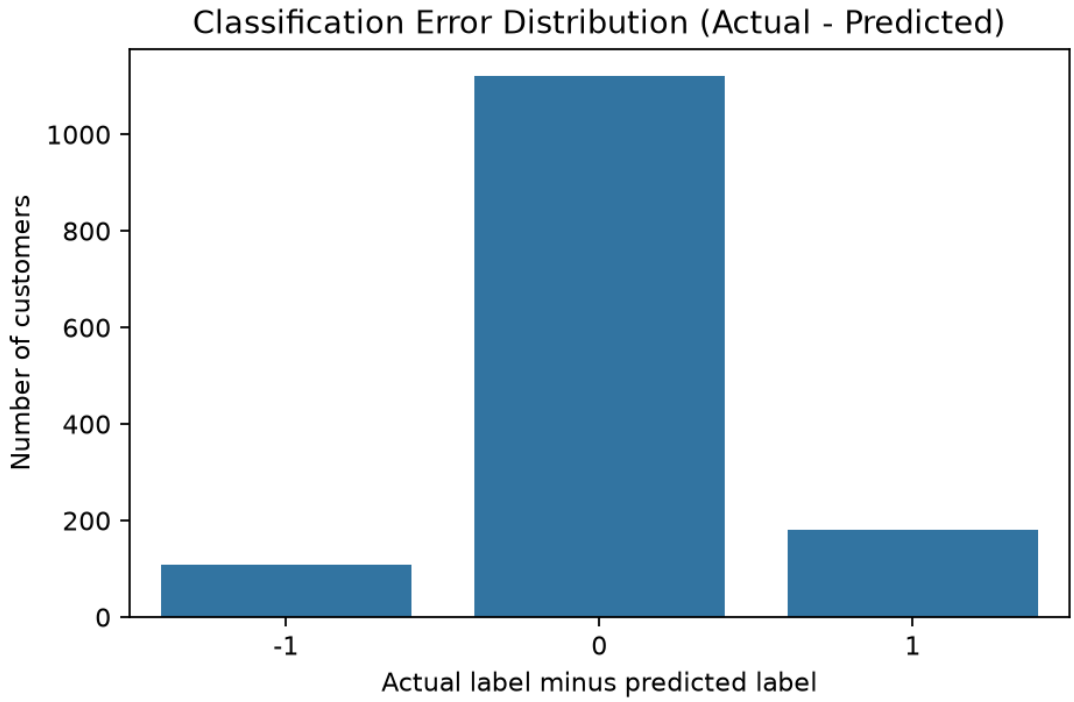


Inserted chart: 08_roc_curve.png

The ROC curve plots the true-positive rate against the false-positive rate at different classification thresholds. The test ROC-AUC was 0.8376.

Plots true-positive rate against false-positive rate across thresholds and reports test ROC-AUC.

Figure 9 — Error distribution

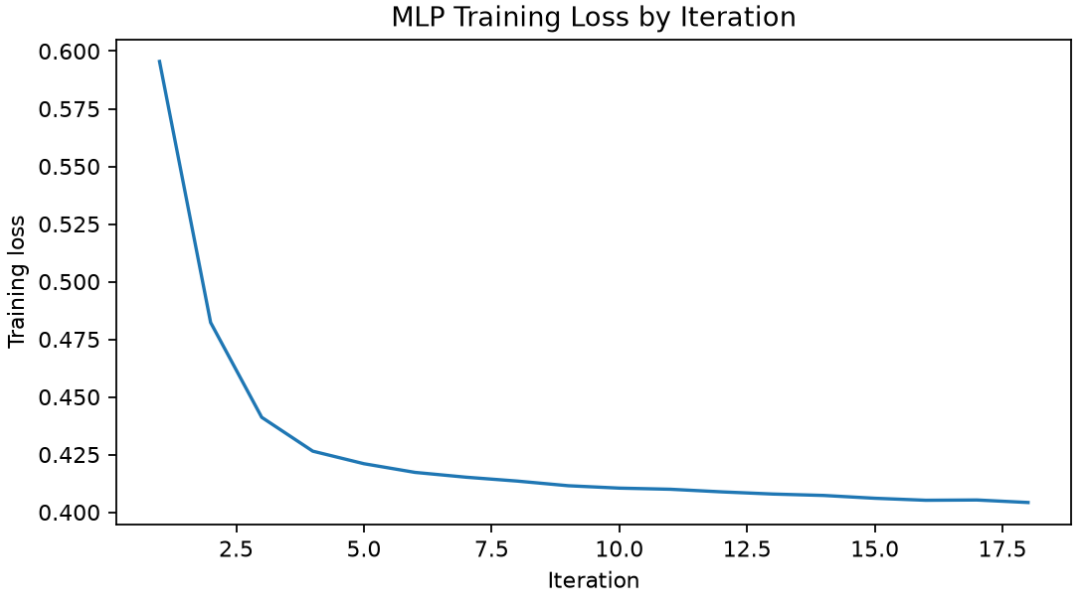


Inserted chart: 09_error_distribution.png

The error distribution summarizes classification mistakes using actual labels minus predicted labels.

Summarizes classification mistakes using actual minus predicted labels.

Figure 10 — Training loss



Inserted chart: 10_training_loss.png

The loss curve shows how the neural network's training loss changed over iterations. The curve should be interpreted alongside held-out test metrics.

Shows the MLP training loss over iterations. A flattening curve suggests the optimizer is making smaller improvements; the curve alone does not prove generalization.

After running the code, copy the actual values from outputs/metrics.json into the following table. This report intentionally does not invent model results.

Metric	Observed test result (fill after run)
Accuracy	79.56%
Precision	64.24%
Recall	51.87%
F1-score	57.40%
ROC-AUC	0.8376
Best validation ROC-AUC	0.8468
Selected hidden layers and alpha	Hidden layers (32, 16); alpha = 0.0001

Observed Results from Execution

The executed program loaded 7,043 customer records and 21 columns. The dataset contained 11 missing values in TotalCharges and no duplicate rows. Churn distribution was 5,174 No Churn records and 1,869 Churn records. The average tenure was approximately 32.37 months, and average MonthlyCharges was approximately 64.76 dataset currency units.

The best validation ROC-AUC among the tested configurations was 0.8468, obtained with hidden layers (32, 16) and alpha = 0.0001. On the held-out test set of 1,409 records, the model achieved 79.56% accuracy, 64.24% precision for the Churn class, 51.87% recall for the Churn class, 57.40% F1-score for the Churn class, and ROC-AUC of 0.8376. Recall indicates that the model detected about half of actual churners, so improving churn detection is a useful future direction.

7. Conclusion

This project establishes a reproducible workflow for examining telecom customer churn through EDA, statistical summaries, visualizations, and an MLP classifier. The charts help compare churn groups across tenure, monthly charges, contract types, and internet service. The model is evaluated on held-out data using several complementary metrics rather than accuracy alone. The final conclusion about model quality and the strongest observed patterns should be completed after the generated figures and metrics have been reviewed.

7.1 Practical Recommendations

- Use the EDA findings to identify customer groups that may merit further investigation, rather than treating association as proof of cause.
- Evaluate recall and precision together before choosing a churn intervention threshold.
- Test any retention strategy with a controlled business evaluation before deploying it.

7.2 Limitations and Future Scope

The dataset is a sample dataset and may not represent every telecom market or current customer population. Some fields are categorical summaries and the dataset does not establish causal relationships. Future work could compare the MLP with logistic regression and tree-based models, tune classification thresholds using business costs, calibrate predicted probabilities, add explainability methods, and validate on newer data.

8. References

1. IBM. (n.d.). Telco Customer Churn sample dataset. https://github.com/IBM/telco-customer-churn-on-icp4d
2. scikit-learn developers. (n.d.). MLPClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
3. scikit-learn developers. (n.d.). Model evaluation: scoring, metrics, and classification. https://scikit-learn.org/stable/modules/model_evaluation.html
4. pandas development team. (n.d.). pandas documentation. https://pandas.pydata.org/docs/
5. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Appendix A — Source Code

The complete, commented Python implementation is provided separately at src/telecom_churn_eda.py. It performs data loading, preprocessing, EDA, MLP training, evaluation, and chart generation. Include the source code in the GitHub repository as required by the assignment..